

Abhishek M R — Personal Viva Defense Handbook

Role: Frontend Architecture, Clinical UI Design & Full-Stack System Integration

Student Name: Abhishek M R	University Seat No (USN): 20231CSE0268
Assigned Role: Frontend & System Integration	Project Guide: Mr. Jetti Satya Sai Kumar
Project Title: MindBridge: Child Mental Health Surveillance	Institution: Presidency University, Bengaluru (2026–2027)

1. Your 30-Second Opening Statement to the Examiners

"Good morning, respected examiners. My name is **Abhishek M R**, and my core contribution to MindBridge was **Frontend Engineering & Full-Stack System Integration**. I built the responsive single-page React architecture, designed the child-friendly daily check-in and standardized PSC-17 pediatric assessment interfaces, engineered the dual-mode API integration layer with FastAPI, and optimized the client performance to achieve a **92/100 Lighthouse Performance score** and **0 ms Total Blocking Time**."

2. Deep-Dive Breakdown of Your Assigned Areas

A. React Frontend Architecture & Route Code-Splitting

- **Framework & Bundler:** Developed using **React 19** with **Vite 8** (Rolldown engine) for sub-second development builds and optimized production tree-shaking.
- **Design System:** Engineered a bespoke, institutional flat design system with Tailwind CSS (1px crisp borders, zero glow effects, high contrast).
- **Dynamic Code-Splitting:** Implemented `React.lazy()` and `Suspense` on all major views (ChildAssessment, GuardianDashboard, Diagnostics, CopingToolkit, CrisisResources, Auth). Reduced initial JavaScript bundle from **774 KB** down to **283 KB** (85.7 KB gzipped) with a **1.4s First Contentful Paint**.

B. Child Daily Mood & Lifestyle Check-in Interface (CheckIn.jsx)

- **Cognitive Load Reduction:** Implemented 6 single-tap affective emoji anchors with descriptive helper subtitles (Joyful = 6, Calm = 5, Neutral = 4, Worried = 3, Sad = 2, Frustrated = 1).
- **Lifestyle Sliders:** Built 4 interactive range sliders with real-time numeric readouts and health benchmarks: Sleep (4–12h, 9–11h ideal), Screen Exposure (0–8h, ≤2h recommended), Physical Play (0–4h, ≥1h recommended), and School Stress (1–5 scale).
- **In-Memory Journal (DPDP Act 2023 Section 9):** Configured volatile RAM-only sentiment processing—journal reflections are scored for VADER compound valence in-memory and **never persisted to database disk**.
- **Diagnostic Card:** Renders real-time Random Forest risk tier (0/1/2), Gini factor attribution explanation, VADER sentiment valence, and supportive affirmations.

C. Standardized Pediatric Assessment Interface (ChildAssessment.jsx)

- **Clinical Pedigree (PSC-17):** Built the validated 17-item Pediatric Symptom Checklist screener (Gardner et al., 1999) using a 3-point Likert scale (Never = 0, Sometimes = 1, Often = 2).
- **Subscale Tab Filtering:** Category filters for Internalizing (Q1–Q5), Attention (Q6–Q10), Externalizing (Q11–Q14), and Social Connectedness (Q15–Q17).
- **Clinical Cutoff Scoring Logic:** Automatically computes total score (max 34) and evaluates against the clinical cutoff (≥ 15). Flags subscale elevations: Internalizing $\geq 5/10$, Attention $\geq 7/10$, Externalizing $\geq 7/8$.
- **Evaluator Presets & History:** Integrated 1-click test fillers (Normative Low, Elevated Distress, Focus Strain), printable clinical summary report, and SQLite longitudinal history drawer.

D. Frontend-to-Backend API System Integration (api.js)

- **RESTful Gateway:** Structured async fetch API client communicating with FastAPI endpoints over HTTP/JSON with CORS middleware headers.
- **Dual-Mode Standalone Fallback:** Engineered client-side machine learning rules (evaluateClientCheckin) and psychometric scoring (evaluateClientAssessment) with localStorage persistence. Ensures 100% functionality on static cloud hosts (Vercel) even when the backend is offline.
- **Health Heartbeat:** Background polling (15s interval) controlling a live API status dot in the navigation header.

E. UI/UX Polishing, Performance Optimization & Debugging

- **Lighthouse Audit Score:** Performance: 92/100 | Best Practices: 100/100 | SEO: 100/100 | Total Blocking Time: 0 ms.
- **Font Optimization:** Replaced blocking CSS @import with preconnected, preloaded asynchronous font headers in index.html.
- **Persona Hub:** Standardized state management for pre-configured roles: **Ashrith** (Child, Age 11), **Nitiz** (Guardian / Parent), and **Dr. Arjun Cement** (Clinician / Counselor).

3. Abhishek's Quick Reference Technical Cheat Sheet

Parameter / Component	Value / Implementation	Technical & Academic Details
Frontend Framework	React 19 + Vite 8	Single-Page Application (SPA) with declarative component architecture
Lighthouse Metrics	Perf: 92, SEO: 100, TBT: 0ms	FCP: 1.4s, LCP: 1.5s, Speed Index: 1.4s (Production build)
Bundle Optimization	283 KB (85.7 KB gzip)	Route-level lazy loading via React.lazy and Suspense
PSC-17 Total Cutoff	Total Score ≥ 15 / 34	Gardner et al. (1999) clinical threshold for psychosocial consultation
PSC-17 Subscale Cutoffs	Int $\geq 5/10$, Att $\geq 7/10$, Ext $\geq 7/8$	Internalizing (Emotional), Attention (ADHD), Externalizing (Conduct)
Privacy Baseline	DPDP Act 2023 (Section 9)	Child reflections analyzed in-memory only; raw text never written to disk
API Client Mode	Dual-Mode (Online / Fallback)	FastAPI REST over HTTP with seamless LocalStorage client fallback
Demo Personas	Ashrith, Nitiz, Dr. Arjun Cement	Pre-seeded role catalog for instant 1-click evaluator demonstrations

4. Master Reviewer Q&A Defense (Top 10 Questions for Abhishek)

Here are the exact questions examiners will ask you regarding frontend and system integration, with model answers:

Q1: What was your specific, individual contribution to this project?

Ans: I was responsible for Frontend Engineering and Full-Stack System Integration. I built the entire React web application, created the Child Mood Check-in and PSC-17 Pediatric Assessment interfaces, integrated the client with the FastAPI REST backend via asynchronous API endpoints, implemented the dual-mode client fallback logic, and optimized performance to achieve a 92/100 Lighthouse score.

Q2: Why did you choose React and Vite instead of vanilla HTML or Next.js?

Ans: We chose React for its component reusability and declarative state management, which is essential for dynamic UI elements like multi-slider lifestyle inputs, dynamic charting, and questionnaire scoring. We used Vite because its native ES-module dev server and Rollup production bundler build our entire application in under 750ms with automatic tree-shaking.

Q3: How does the Child Check-in interface minimize cognitive load for young children?

Ans: Children aged 6–17 cannot be expected to fill long clinical forms. In CheckIn.jsx, I implemented single-tap affective emoji anchors with descriptive subtitles and intuitive horizontal range sliders with health guidelines (e.g., '9-11h ideal sleep'). The child can complete a full self-report in under 30 seconds.

Q4: How does the frontend handle clinical cutoff scoring for PSC-17?

Ans: In ChildAssessment.jsx, 17 questions are mapped across 4 subscales. Each question is scored on a 3-point Likert scale (0 for Never, 1 for Sometimes, 2 for Often). When submitted, the system computes the total score (max 34). If total score is ≥ 15 , or if the internalizing subscale is ≥ 5 , or attention ≥ 7 , it flags the respective domain and displays targeted psychoeducational recommendations.

Q5: How does your frontend ensure data privacy under the DPDP Act 2023?

Ans: Under Section 9 of the DPDP Act, children's personal reflections cannot be stored detrimentally. In the frontend, the reflection journal is explicitly marked as 'in-memory only'. When sent to the backend, it is evaluated for sentiment compound in volatile RAM and immediately discarded—it is never written to disk or displayed on the Guardian Dashboard.

Q6: How did you connect the React frontend with the FastAPI backend?

Ans: In frontend/src/api.js, I created modular async wrapper functions using standard fetch. The backend is configured with FastAPI CORS middleware to accept requests from our client. We pass structured JSON payloads validated against Pydantic schemas on the server and receive strongly-typed JSON responses.

Q7: What happens if the Python backend goes down during a live demo?

Ans: I built a dual-mode client fallback in api.js. If fetch fails or the backend is offline, the client automatically catches the error and runs client-side psychometric scoring (evaluateClientAssessment) and lifestyle risk estimation using localStorage. The user experiences zero crashes or blank screens.

Q8: How did you optimize the frontend performance for Google Lighthouse?

Ans: I did three key things: (1) Implemented route-level code splitting using React.lazy and Suspense, dropping initial JS payload from 774 KB to 283 KB; (2) Removed render-blocking @import CSS font calls and replaced them with preconnected DNS prefetch headers in index.html; (3) Added standard robots.txt and semantic metadata, bringing our SEO to 100/100 and Performance to 92/100 with 0 ms Total Blocking Time.

Q9: How are the longitudinal charts rendered on the Guardian Dashboard?

Ans: We use Recharts, a composable React charting library built on SVG. It renders an AreaChart for daily mood trajectories, a LineChart for the 7-day rolling average, a PieChart for risk tier distribution, and a multi-line chart correlating sleep, screen time, physical play, and school stress.

Q10: How is user authentication and persona switching managed in the UI?

Ans: In Auth.jsx and Navbar.jsx, we maintain active user state using React state synced with localStorage. Users can switch between 3 pre-seeded demo personas (Ashrith for Child, Nitiz for Guardian, Dr. Arjun Cement for Clinician) or register a custom account with role-specific views.